# Vorbereitung auf die Bergtour

2024 wurde von den Teilnehmern der Österreichischen Informatikolympiade nach mehreren Fehlversuchen erstmals der Möslalmkogel (1109 m) bei Wörgl erklommen. 2025 hat das österreichische IOI-Team in Bolivien dann den Cerro Churuquella (3.116 m) erklommen. 2026 soll das in Usbekistan noch getoppt werden. Dafür braucht es jedoch einiges an Vorbereitung, da die Bergtouren in Usbekistan selbst für österreichische Verhältnisse anspruchsvoll sind. Also nicht mehr ganz eine "nette Feierabendrunde"[1], und man kann da nicht mehr sagen: "Wirklich schwierig ist hier nichts"[2]. Deshalb beginnt die Vorbereitung natürlich wieder mit dem Möslalmkogel. Danach sollen noch drei weitere Berge in aufsteigender Höhe erklommen werden. Um Fahrzeit zu sparen, muss der zweite Berg östlicher liegen als der erste, und der dritte Berg östlicher als der zweite. Es soll also eine Bergtour mit drei Bergen geplant werden, die von Westen nach Osten immer höher werden.

Zum Glück gibt es für diese Bergtouren viele Möglichkeiten. Wie viele genau es gibt, sollst du herausfinden. Dafür haben wir dir eine Konstruktionsbeschreibung für die Liste der Berghöhen gegeben. Mit den folgenden $n$ Schritten kannst du diese Liste konstruieren. Zu Beginn haben alle Berge die Höhe 0 (die Höhe des Möslalmkogels). In jedem Schritt $i$ ($1 \leq i \leq n$) wird die Höhe eines Intervalls an Bergen erhöht. Genauer gesagt werden die Höhen aller Berge von $a_i$ bis $b_i$ inklusive um eine Einheit erhöht. Am Ende der $n$ Schritte hast du eine Liste mit den Höhen aller Berge. Alle für die Aufgabe relevanten Berge haben einen Index $\geq 0$.

## Implementierung

Deine Aufgabe ist es, die Prozedur `int calculate(intervals)` zu implementieren.

```
int calculate(vector<pair<long, long>> &intervals)
```

Die Funktion wird einmal zu Beginn aufgerufen und erhält die Liste der Intervalle.

- `intervals` ist ein Vektor von Paaren aus `long`, wobei jedes Paar $a_i$ und $b_i$ das Intervall der Berge angibt, deren Höhe in Schritt $i$ erhöht wird.
- Die Funktion soll die Anzahl der Möglichkeiten zurückgeben, drei Berge auszuwählen, die von Westen nach Osten immer höher werden. Da diese Zahl sehr groß sein kann, soll sie modulo $10^9 + 9$ zurückgegeben werden. Wenn es keine Möglichkeit gibt, soll $-1$ zurückgegeben werden.

## Beispiele

[1] Andy84
[2] Andy84

| Eingabe | Ausgabe | Anmerkungen |
|---|---|---|
| 3<br>1 1<br>2 3<br>2 3 | 2 | Mögliche Bergtouren: (0, 1, 2) und (0, 1, 3) |

| Eingabe | Ausgabe | Anmerkungen |
|---|---|---|
| 3<br>0 0<br>2 3<br>2 3 | -1 | Hier gibt es keine möglichen Bergtouren. |

| Eingabe | Ausgabe | Anmerkungen |
|---|---|---|
| 2<br>1000000009<br>1000000010<br>1000000010<br>1000000010 | 0 | Hier gibt es exakt $10^9 + 9$ mögliche Bergtouren, aber da die Antwort modulo $10^9+9$ zurückgegeben werden soll, ist die Antwort 0. |

| Eingabe | Ausgabe |
|---|---|
| 6<br>1 4<br>2 4<br>6 8<br>8 8<br>6 6<br>6 6 | 13 |

## Subtasks

Allgemein gilt:

- $0 \leq a_i, b_i \leq 10^{18}$
- $1 \leq n \leq 10^5$

**Subtask 1 (17 Punkte):** $0 \leq a_i, b_i, n \leq 100$

**Subtask 2 (15 Punkte):** $n \leq 100$

**Subtask 3 (13 Punkte):** $0 \leq a_i, b_i, n \leq 3000$

**Subtask 4 (11 Punkte):** $n \leq 3000$

**Subtask 5 (19 Punkte):** $0 \leq a_i, b_i, n \leq 10^5$

**Subtask 6 (25 Punkte):** Keine Einschränkungen

## Limits

**Zeitlimit:** 1 s **Speicherlimit:** 256 MB

## Eingabe

Das Einlesen der Eingabe wird von einem von uns geschriebenen Programm, dem Grader, übernommen. In der ZIP-Datei zu dieser Aufgabe findest du eine Grader-Datei, die du zum Testen deiner Lösung mit deinem Programm kompilieren kannst.

Dieser Grader liest die Eingabe im folgenden Format ein: Die erste Zeile enthält eine Zahl $n$, die Anzahl der Intervalle, die die Höhen der Berge erhöhen. Die nächsten $n$ Zeilen enthalten jeweils zwei Zahlen $a_i$ und $b_i$, die die Intervalle der Berge angeben, deren Höhe in Schritt $i$ erhöht wird.

| **Eingabe** | **Ausgabe** |
|---|---|
| `n`<br>$a_1$ $b_1$<br>$a_2$ $b_2$<br>$\vdots$<br>$a_n$ $b_n$ | *result* |